

Notes on Function Approximators

1 Motivation

State aggregation and tile coding (see the continuous-observation-space notes) turn a continuous state into a discrete index and keep an explicit table entry per cell — every cell is still learned *independently*. **Function approximation** goes one step further: $\hat{v}$ (or $\hat{q}$) is a parametric function of the state,

$$\hat{v}(s \mid \mathbf{w}) \approx v_\pi(s), \quad \mathbf{w} \in \mathbb{R}^n,$$

with n typically far smaller than $|\mathcal{S}|$ (or, for continuous $\mathcal{S}$, finite instead of uncountable). Updating $\mathbf{w}$ at one state now changes the estimate at *every* state whose features overlap with it — this is exactly the generalization tile coding hinted at, taken to its natural conclusion. The price is that $\hat{v}$ can no longer represent every value function exactly; it is restricted to whatever the parametric family can express.

2 The prediction objective

With a table, "best fit" was unambiguous: exact per-state values. With function approximation, $\mathbf{w}$ has fewer dimensions than $\mathcal{S}$, so improving the estimate at one state can worsen it at another. We need an explicit objective. The standard choice is the **Mean Squared Value Error**, weighted by how often each state is visited under the data-generating policy, $\mu(s)$:

$$\overline{VE}(\mathbf{w}) = \sum_{s \in \mathcal{S}} \mu(s) \left[v_\pi(s) - \hat{v}(s \mid \mathbf{w}) \right]^2.$$

Since $\mathcal{S}$ (or μ) is generally unknown or intractable to sum over exactly, $\mathbf{w}$ is instead adjusted by **stochastic gradient descent (SGD)** on individual, sampled states — each visited state S_t nudges $\mathbf{w}$ in the direction that reduces the squared error *at that state*, in proportion to how often such states occur under μ automatically, simply because they are sampled with that frequency.

3 Gradient descent for value prediction

Suppose at time t we have a target U_t for $\hat{v}(S_t \mid \mathbf{w})$ (the true $v_\pi(S_t)$ if it were available, or — as in the algorithms below — a sampled/bootstrapped stand-in for it, such as the Monte Carlo return G_t or a TD target). The per-example squared error is $\frac{1}{2}[U_t - \hat{v}(S_t \mid \mathbf{w})]^2$, and the SGD update moves $\mathbf{w}$ against its gradient:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \left[U_t - \hat{v}(S_t \mid \mathbf{w}_t) \right] \nabla \hat{v}(S_t \mid \mathbf{w}_t),$$

where $\nabla \hat{v}(s | \mathbf{w}) = \left(\frac{\partial \hat{v}(s|\mathbf{w})}{\partial w_1}, \dots, \frac{\partial \hat{v}(s|\mathbf{w})}{\partial w_n} \right)^\top$. If U_t itself depends on $\mathbf{w}_t$ (as it does for TD targets, which bootstrap off $\hat{v}$), this is only an approximation to the true gradient of $\overline{VE}$ — hence **semi-gradient** methods (Section 5): they differentiate through $\hat{v}(S_t | \mathbf{w})$ but treat the target U_t as fixed, ignoring its own dependence on $\mathbf{w}$.

4 Linear function approximation

The most important special case represents $\hat{v}$ as a **linear** function of a hand- or auto-designed feature vector $\phi(s) \in \mathbb{R}^n$:

$$f(s | \mathbf{w}) = \hat{v}(s | \mathbf{w}) = \mathbf{w} \cdot \phi(s)^\top = \sum_{i=1}^n w_i \phi_i(s).$$

Because $\hat{v}$ is linear in $\mathbf{w}$, its gradient is simply the feature vector itself, $\nabla \hat{v}(s | \mathbf{w}) = \phi(s)$, so the general SGD update of Section 3 reduces to the especially simple

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \left[U_t - \hat{v}(S_t | \mathbf{w}_t) \right] \phi(S_t).$$

For linear methods, $\overline{VE}$ is a quadratic (bowl-shaped) function of $\mathbf{w}$ with a single minimum, so gradient and semi-gradient methods have much stronger convergence guarantees than with nonlinear approximators such as neural networks (Section 7). Everything therefore hinges on the choice of feature map ϕ .

5 Constructing features: bases for linear methods

5.1 Polynomial basis

Given a raw (already numeric) state vector $s = (s_1, s_2, \dots, s_k)$, the **polynomial basis** augments the state with powers and cross-terms of its components, letting a *linear* model in the augmented features capture nonlinear relationships in the original state:

$$\phi(s) = [s_1, s_1^2, \dots, s_1^k, s_2^i, s_3^i, \dots, s_n],$$

with parameter vector

$$\mathbf{w} = [w_1, w_2, w_3, \dots, w_n],$$

so that the approximator expands to an ordinary polynomial in the state components:

$$f(s) = \hat{v}(s) = w_1 \cdot s_1 + w_2 \cdot s_1^2 + w_3 \cdot s_2 + \dots$$

Each additional power or cross-term is just one more entry of $\phi(s)$ and one more weight to learn — e.g. including s_1^2 lets the approximator represent curvature along dimension 1 that a purely linear term $w_1 s_1$ could not, at the cost of one extra parameter. The degree of the polynomial (how high the powers go, and whether cross-terms like $s_1 s_2$ are included) controls the usual expressiveness/data-efficiency tradeoff: higher degree can fit more complex v_π , but has more weights to estimate from the same amount of experience.

5.2 Fourier basis

Instead of powers, use cosines of the (rescaled-to-[0, 1]) state:

$$\phi_i(s) = \cos(\pi \mathbf{c}_i \cdot s), \quad \mathbf{c}_i \in \{0, 1, \dots, K\}^k,$$

for integer coefficient vectors $\mathbf{c}_i$ up to some order K per dimension. Like the polynomial basis this is a fixed, hand-designed feature map used linearly, but the Fourier basis tends to behave better numerically (its terms are bounded in $[-1, 1]$, unlike s_1^k which can blow up) and is a standard default for low-dimensional continuous control tasks.

5.3 Tile coding, revisited

Tile coding (previous notes) is also linear function approximation: its feature vector $\phi(s) \in \{0, 1\}^{n \cdot \prod_i b_i}$ has exactly one active ("1") entry per tiling and zero elsewhere, so $\hat{v}(s \mid \mathbf{w}) = \mathbf{w} \cdot \phi(s)$ simply sums the n active weights — equivalent to the "average of n table lookups" view used earlier. Unlike the polynomial or Fourier basis, tile-coding features are *local* (only nearby states share active tiles), which tends to make learning faster and more stable, at the cost of coarser global extrapolation.

6 Semi-gradient TD(0) and gradient Monte Carlo

Two concrete instantiations of the update in Section 3/4, differing only in what stands in for U_t :

Algorithm 1 Gradient Monte Carlo for $\hat{v} \approx v_\pi$

```

1: Initialize  $\mathbf{w}$  arbitrarily (e.g.  $\mathbf{0}$ ); fix  $\alpha$ 
2: for episode = 1, 2, ... do
3:   Generate a full episode  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$  following  $\pi$ 
4:   for  $t = 0, 1, \dots, T - 1$  do
5:      $G_t \leftarrow$  return from step  $t$  onward
6:      $\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t \mid \mathbf{w})] \nabla \hat{v}(S_t \mid \mathbf{w})$ 
7:   end for
8: end for
```

Since $U_t = G_t$ does not depend on $\mathbf{w}$, this is a *true* gradient method and inherits SGD's standard convergence guarantee to a local (for linear $\hat{v}$, global) minimum of $\bar{V}E$.

Algorithm 2 Semi-gradient TD(0) for $\hat{v} \approx v_\pi$

```

1: Initialize  $\mathbf{w}$  arbitrarily; fix  $\alpha$ 
2: for episode = 1, 2, ... do
3:   Initialize  $S_0$ 
4:   for  $t = 0, 1, 2, \dots$  until  $S_t$  terminal do
5:     Take  $A_t \sim \pi(\cdot \mid S_t)$ , observe  $R_{t+1}, S_{t+1}$ 
6:      $U_t \leftarrow R_{t+1} + \gamma \hat{v}(S_{t+1} \mid \mathbf{w})$  (0 if  $S_{t+1}$  terminal)
7:      $\mathbf{w} \leftarrow \mathbf{w} + \alpha [U_t - \hat{v}(S_t \mid \mathbf{w})] \nabla \hat{v}(S_t \mid \mathbf{w})$ 
8:   end for
9: end for
```

Here U_t bootstraps off $\hat{v}(S_{t+1} \mid \mathbf{w})$, so it silently depends on the very $\mathbf{w}$ being updated — the true gradient of $\overline{VE}$ would need to account for that dependence, but the update above only follows $\nabla \hat{v}(S_t \mid \mathbf{w})$ and ignores it (hence *semi*-gradient). It converges reliably for linear $\hat{v}$ (to a fixed point near, though not exactly at, the $\overline{VE}$ minimum), but loses even that guarantee for general nonlinear approximators combined with bootstrapping and off-policy data — the well-known **deadly triad** of function approximation, bootstrapping, and off-policy learning.

7 Extending to control: $\hat{q}(s, a \mid \mathbf{w})$

Everything above generalizes directly to action values by featurizing state-action pairs, $\phi(s, a)$, and approximating

$$\hat{q}(s, a \mid \mathbf{w}) = \mathbf{w} \cdot \phi(s, a)^\top,$$

with the same semi-gradient update but plugging in a control target (e.g. the Sarsa target $R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1} \mid \mathbf{w})$, or the Q-learning target $R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a' \mid \mathbf{w})$) in place of U_t , and an ε -greedy (or otherwise soft) policy over $\hat{q}(S_t, \cdot \mid \mathbf{w})$ in place of the tabular action-selection step. This is the bridge from tabular/tile-coded Sarsa to modern approaches such as DQN, where $\hat{q}(s, a \mid \mathbf{w})$ is a (nonlinear) neural network instead of a linear-in-features model.

8 Linear vs. nonlinear approximators

	Linear (poly./Fourier/tile)	Nonlinear (neural network)
Form	$\mathbf{w} \cdot \phi(s)^\top$, fixed hand-designed ϕ	$\hat{v}(s; \theta)$, features learned jointly with weights
$\overline{VE}$ landscape	convex (single minimum)	generally non-convex
Semi-gradient TD convergence	guaranteed (on-policy)	not guaranteed; deadly triad risk with bootstrapping + off-policy
Feature engineering effort	manual (choose basis, resolution)	largely automatic (representation learned)
Typical scale	low-dimensional continuous control	high-dimensional (pixels, large state spaces)

9 Reference

Sutton, R. S., & Barto, A. G. *Reinforcement Learning: An Introduction*, 2nd ed., Ch. 9 (On-policy Prediction with Approximation, esp. 9.3–9.5 on gradient methods and polynomial/Fourier/tile-coding bases), Ch. 10 (On-policy Control with Approximation), and Section 11.3 (The Deadly Triad).